

# Data reduction and feature generation techniques for classification algorithms on heterogenous behavioural data

Marko Gavrić<sup>1\*</sup>, Mislav Zorko<sup>1</sup> and Siniša Slijepčević<sup>1</sup>

<sup>1\*</sup>Cantab Predictive Intelligence, St John’s Innovation Centre,  
Cambridge, CB40WS, UK.

\*Corresponding author(s). E-mail(s): [mgavric@cantabpi.com](mailto:mgavric@cantabpi.com);  
Contributing authors: [mzorko@cantabpi.com](mailto:mzorko@cantabpi.com); [sinisa@cantabpi.com](mailto:sinisa@cantabpi.com);

## Abstract

We introduce an abstract framework for developing accurate classification machine learning models on a combination of certain time series and tabular data sources occurring frequently in practice. We generalise use cases such as predicting customer engagement (e.g. an email click), likelihood to buy a specific product on a digital platform, and behavioural credit scoring (i.e. likelihood to default); develop an abstract framework for generating features from data which enables capturing of expert knowledge; and then proceed with optimising the model training and validation procedure. We also give actual examples of the model performance, evolution of the model performance, and the feature importance on actual production data.

**Keywords:** Machine learning, Artificial Intelligence, Classification, Feature generation, Model optimisation, AUC, Model performance

## 1 Introduction

Feature generation is an important step in processing raw data and generating potentially more complex data variables ("features") which are the actual inputs to machine learning and artificial intelligence (ML/AI) models. The arguably dominant paradigm is that feature generation should be automated and scalable, e.g. either by deploying dedicated neural network models [1–3], machine learning models [4], or by deploying

standardised numerical procedures provided as dedicated packages in R, Python, and commercial platforms such as DataRobot and H2O [5–7]. Nevertheless, as we also show in this paper, the automated feature generation procedure often results in model under-performance for multiple reasons discussed in detail in the final section.

We argue and demonstrate in this paper that for many use cases appearing frequently in applications, the feature generation procedure should be adjusted to the use case at hand, and that it should enable effective incorporation of the expert knowledge. The key example of such an approach is the AlphaFold family of algorithms for predicting protein’s 3D structure from its amino acid sequence [8–10]. The feature generation procedure is in that case very sophisticated and use-case specific: it incorporates a detailed mathematical, biochemical and physical knowledge of the problem at hand by deploying rather advanced mathematical and statistical apparatus. Furthermore, as the data set for model training contains ‘only’ roughly 300,000 proteins with the known 3d structure, it is very unlikely that training an ML/AI model without such a specific feature generation step would be nearly as effective. [11], [12] also welcome a bespoke feature generation approach.

In this paper, we focus on a family of classification problems frequently appearing in the business practice, described in detail in Section 2. The common characteristic of these use cases is that the key data source describes historical behaviour for a certain set of *customers*, with recorded history of the *triggers* (e.g. emails sent to the customer), *events* (views of certain content on the digital platform) or *transactions* (e.g. purchases). We give an abstract framework which covers the structure of all these data sources. Furthermore, in each of these use cases, the number of customers (thus the number of data points used for model training), typically does not exceed single millions, and may be one or two orders of magnitude less than that.

Our key contribution is an introduction of a scalable, semi-automated approach for feature generation for this class of models. The semi-automation step is designed to enable incorporating of specific expert, for example business knowledge, by selecting appropriate *statistics* and *configuration parameters* for each feature class [13]. We describe the detailed approach to constructing features in Section 4, to modeling and feature selection in Section 5, and the technical approach in Section 7. We demonstrate that the proposed feature generation approach is critical for model accuracy by showing examples of actual models in production in Section 8. In each of the considered cases, our feature generation approach dramatically improved model accuracy as compared to deploying only automated, ‘off-the-shelf’ feature generation procedures. In each of the considered cases, the choice of the actual model, and contribution of such model steps as model parameter fine-tuning, contributed less to the model accuracy than the choice of the appropriate feature generation procedure.

Finally, we give a high-level description of how these models are deployed in actual production, and summarise learnings in Section 9.

## 2 The abstract data structure

The abstract data structure described in this section is common for a variety of use cases occurring typically in Business to Business (B2B) or Business to Consumer (B2C)

sales and marketing. They typically contain information on historical behaviour of a smaller or larger pool of customers when targeted through different marketing channels in the past. This information may be in the form of the type of channel, message or similar, or in the form of customer’s reaction, such as purchase, type of purchase, or engagement (such as opening or clicking on an email). The use cases cover both physical channels (such as sales through a large pool of sales representatives), direct digital channels (e.g. email or push mobile notifications), sales via a digital platform, and the omni-channel approach combining all these channels.

The abstract data structure is given in Tables 1, 2, 3.

The first data source type generalises past events. Typically there are multiple tables as given in Table 1 of the same generic structure corresponding to different sales channels. It is important that the trigger segmentation variables (*Trigger\_segment\_i* variables) are **common** for all the data sources, while the event response variables (*Response\_i*) may depend on the channel (for simplicity we omitted the channel dependency in the names of the response variables). We give a number of examples from actual use cases below.

The second data source type given in Table 2 gives various customer segmentation. The customer segmentation may be demographic (e.g. gender, marital status, etc.), location data, or any other data inferred from alternative data sources [14].

Finally, in Table 3 we give the historical data source of *Outcomes*, which we want to model, predict, and ultimately optimise. As described below, these outcomes may be related to customer engagement (e.g. predicting if a customer will click on an email), customer purchase following a marketing trigger, or even (as given in Example 3 below), a historical information on customer’s default on a cash loan in retail banking. The outcome here is given by the variable *Label*, of the Boolean type, predicted by a classification machine learning model which is the focus of this paper.

There are several intricacies related to this data structure which we cover in more detail later in this paper, but introduced now.

Firstly, the structure of the training data and the data in actual production subtly differ. In the training data, the *Time* variable for both events and outcomes variables. In the actual production data, the historical events data is essentially the same data source, while for outcomes, Time is typically now (or some fixed, known time in the near future, when the actual marketing or equivalent action is planned).

Secondly, when preparing the data for modelling, we need to pay careful attention to include only events occurring prior to the actual outcome. More specifically, when calculating features associated to a specific historical outcome, we must rigorously filter and use only event data for events occurring in the past, as related to that specific outcome. This is one of the key underlying themes of this paper.

Thirdly, the model testing and validation must be safeguarded against *data leakage* and carefully thought to prevent it. The data leakage here encompasses the phenomena where the model performance seems to be overestimated, as the actual information from the *test set* is contained in subtle ways in the *train set*, i.e. the data set on which the actual model was trained. The best practice for use cases covered in this paper is to split the train and test data sets both by time and by customer. The time split is typically done so that in the train set, there are only event and outcome data prior

to a certain `split_time`, and in the test set, all the event data can be included, while the outcome data occurring only after that `split_time` is allowed. The customer split ensures that the same customer (identified by the `customer_id`) can not be at the same time in both the train and the test set.

And finally, we note that when preparing the data as specified in Tables 1, 2, 3, it is recommended to include significant amount of judgement and the expert knowledge. For example, we recommend to ensure that there are not too many *Trigger\_segment* variables (ideally not more than 10), with not too many different possible values (also typically not more than 10 for each variable). The same holds for *Customer\_segment* variables. The reason for this is that the typical size of the data set is not large enough, to enable efficient and accurate information extraction and the feature selection, if the number of features is too large. We thus find it the most effective, if the expert knowledge on the problem at hand is introduced when preparing the input data set specified in this section. For example:

- *Trigger\_segment* variables may denote the type of the marketing content (e.g. the content of an email). It is recommended to label all the content in all the marketing channels in a consistent way, with a limited set of labelling categories (captured each by different *Trigger\_segment* variable) and labelling values. The expert knowledge is here required to choose these categories and labels to capture the true differentiators of the customer behaviour, and as much as possible eliminate excess categories and labels not adding value. We also note that the consistent content labelling across marketing channels is implicitly enforced through our data structure, as *Trigger\_segment* variables are common for all the channels.
- *Trigger\_segment* variables may denote the product category being promoted or purchased. Here again we recommend the *less is more* approach: prune typically a huge set of product categories to at most 3, with not more than 10 values in each category.
- *Customer\_segment* variable may contain the customer location. Again, make sure to aggregate location data ideally to not more than 10 aggregate locations. Here the best choice may be non-obvious. For example, it is typically better to aggregate zip codes by a certain feature (e.g. the average income), then to rely on off-the-shelf aggregation to states, counties or equivalent.

We can not emphasize enough the importance of this *less is more* recommendation.

### 3 The key use cases

In this section, we describe three key use cases, including explaining motivation for developing a predictive model, and a description of a mapping of actual use-case specific data to our abstract structure. Further details of deployment of accurate predictive models in production are given in Section 8.

#### Example 1: Predicting engagement

The first example is predicting customer engagement on a specific marketing message [15]. This family of predictive models may be deployed to both business to business

**Table 1** Abstract data structure for events. (One table is given for each sales channel. Trigger segments are common for all channels, while Responses may be channel specific.

| Name              | Key   | Data type          | Description                                        |
|-------------------|-------|--------------------|----------------------------------------------------|
| Event_id          | TRUE  | Character          | Unique id of the event                             |
| Customer_id       | FALSE | Character          | Customer id                                        |
| Time              | FALSE | DateTime           | Timestamp of the event                             |
| Trigger_segment_1 | FALSE | Character          | A subtype of the event ( $1^{st}$ segmententation) |
| ....              |       |                    |                                                    |
| Trigger_segment_k | FALSE | Character          | A subtype of the event ( $k^{st}$ segmententation) |
| Response_1        | FALSE | Numeric or Boolean | Customer's response 1                              |
| ...               |       |                    |                                                    |
| Response_j        | FALSE | Numeric or Boolean | Customer's response j                              |

**Table 2** Abstract data structure for the customer segmentation.

| Name               | Key   | Data type | Description                                         |
|--------------------|-------|-----------|-----------------------------------------------------|
| Customer_id        | TRUE  | Character | Unique id of the event                              |
| Customer_segment_1 | FALSE | Character | Customer segmentation 1 ( $1^{st}$ segmententation) |
| ....               |       |           |                                                     |
| Customer_segment_l | FALSE | Character | Customer segmentation ( $l^{st}$ segmententation)   |

**Table 3** Abstract data structure for outcomes.

| Name              | Key   | Data type | Description                                        |
|-------------------|-------|-----------|----------------------------------------------------|
| Outcome_id        | TRUE  | Character | Unique id of the event                             |
| Customer_id       | FALSE | Character | Customer id                                        |
| Time              | FALSE | DateTime  | Timestamp of the event                             |
| Trigger_segment_1 | FALSE | Character | A subtype of the event ( $1^{st}$ segmententation) |
| ....              |       |           |                                                    |
| Trigger_segment_k | FALSE | Character | A subtype of the event ( $k^{st}$ segmententation) |
| Label             | FALSE | Boolean   | The outcome to be predicted and optimised          |

('B2B') and business to customer ('B2C') targeting by Sales Reps, by direct digital channels or through a digital platform. Accurate prediction of the outcome (e.g. click on an email) may be used in optimisation of marketing messaging, and is valuable for two reasons:

- Marketing channels may be costly (e.g. the time of the Sales rep), thus it is typically important to optimise channel, content and timing of such marketing messages;
- Even in the cases where marketing channels are effectively cost-free (e.g. sending an email), the actual hidden cost is for example the explicit 'opt-out' of a customer (thus loss of the communication channel), or an implicit 'opt-out' (the customer starts ignoring messages as not relevant).

This use case is mapped to the abstract data structure in the following way

- There are multiple 'events' tables, one for each marketing channel (e.g. sales rep visit, email, push notification, etc.). This can also include third-party behavioural data, such as customer's social media activity (where e.g. each tweet, retweet, or like is one event)
- *Trigger\_segment* variables describe the communication content ('content labelling'). As commented above, data engineering (i.e. uniform content labelling across communication channels) is the key data engineering challenge and success factor for this use case. For example, this can also contain the type of the incentive (e.g. the level of discount offered); the influencing technique; etc.
- *Response* variables are one or two Boolean variables, where TRUE denotes successful communication. For some channels, the definition of the Response is straightforward (e.g. opening, and clicking on an email). For other marketing channels, expert knowledge is needed to convert tracking of customer engagement into one or two 'Response' variables with binary outcomes.
- *Customer\_segment* variables contain known demographic information and aggregated location data.
- *Outcome* and *Label* table contain data for a specific planned marketing campaign through a single channel.

## Example 2: Propensity to buy a product

The second example is on predicting likelihood that a customer will purchase a specific product at a specific point in time. This class of models is attempting to predict a specific, inherent customer need, or *utility* for the customer, regardless of a possible promotional approach [16].

The value of such models is rather straightforward: marketers can use this information to tailor message, timing and incentive (e.g. the discount), by e.g. combining the predictions of this class of models with the 'likelihood to engage' class of models from the previous section.

The mapping to the abstract data structure is as follows:

- The events tables represent historical customer interest or purchases for various products. Typically, this can include all past product purchases on a digital platform, all past product views on a digital platform, etc. In addition, retailers often can get access to 2nd or 3rd party data, such as credit card or current account customer transactions (they can be sometimes also obtained through open banking protocols). In such historical transactional data, one can also identify and classify customer purchases in separate 'event' tables, one for each data source [17].
- *Trigger\_segment* data is the product category (typically several product categories)
- *Response* variables contain the purchase amount for purchases or equivalent (a numerical variable), and can be Boolean or omitted for views and similar data.
- *Label* is the Boolean variable denoting a product purchase (yes/no). As typically only product purchases are known, we in practice generate also a set of non-purchases,

by randomly generating 5-10 times non-purchase data, in each instance by a certain criteria 'sufficiently distanced' from all known purchases.

### Example 3: Retail credit risk

Credit risk scoring is a way for banks and other financial institutions to evaluate how likely it is that a person will repay a loan by building scorecards, see [18]. Our approach is to build supervised binary classification models for behavioural credit scoring which predict defaults, i.e., failing to repay the requested loan, based on the Open Banking concept [19]. For this use case, a retail bank can provide anonymised historical customer loan applications, along with applicants' socio-demographics and - most importantly - six months of transactional history data (typically all current account and credit card data).

Historically, the credit risk for retail customers has been estimated on demographic and 'credit history' data only (i.e. aggregated history of customer's credit repayments) [18]. The recent 'open banking' revolution, i.e. the obligation of retail banks to (securely, and subject to customer's consent) share transactional data has provided a new, very rich data source which can significantly improve accuracy of credit scoring models [19].

We focus here on exactly this class of transactional data sources, which can be mapped to our abstract structure as follows:

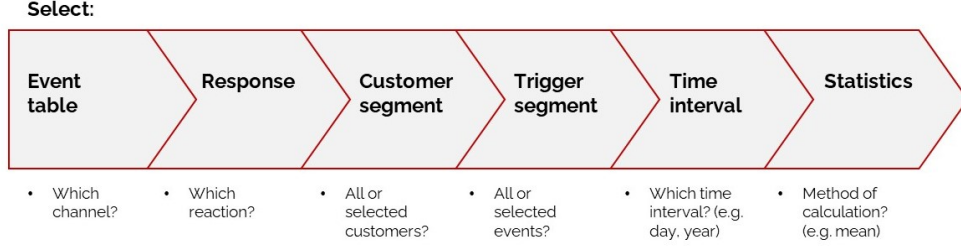
- There are typically two 'event' tables: one for current account, and the other for credit card transactions.
- *Trigger\_segment* variables describe the type or category of the transaction.
- *Response* variables are numeric variables: the transaction amounts.
- *Customer\_segment* variables contain known demographic information and credit history.
- *Outcome* table contains historical credit applications, where *Label* is the ultimate outcome: TRUE if the customer eventually defaulted on the loan. (Note: a critical part of data preparation is 'Reject inference', namely a separate procedure and model to predict - inpute - ultimate outcomes for customers which applied, but did not get a loan. For details see [18] and references therein.)

## 4 Feature generation approach

Our key contribution is a description of a generic feature generation procedure which contains steps as in Figure 1, resulting with a feature table as in Table 4.

The detailed algorithm to construct features is as follows. Each feature is constructed by choosing a single possible choice for a family of categorical or Boolean Configuration Parameters (CPs).

1. **Select channel.** If more than one event ('channel') table is given, a single table is selected based on a specific CP.



**Fig. 1** The schematic representation of the Feature generation algorithm

2. **Select response.** If more than one response is available (e.g. 'open' and 'click' on an email), select a single one.
3. **Select customer segment.** For each *Customer\_segment* field, one of three options is chosen based on a Boolean CP: If  $CP(\text{Customer\_segment}_j)=1$ , only event data for the same *Customer\_segment\_j* as the customer in the corresponding Output table row will be included. If it is FALSE, no event data will be excluded.
4. **Select trigger segment.** The same procedure as for the customer segment is be applied, but now for each Trigger segment. (Note: if the corresponding Trigger segment is not included in the Output table, then the corresponding CP can have values 'all', or a specific Trigger segment value. In the later scenario, only those segments are included.)
5. **Define time interval.** A time interval  $\Delta T$  is chosen, as specified by a CP. In our experience, it is typically optimal to include a limited set of time intervals of different orders of magnitude, such as e.g. hour / day / month / year / all. The chosen interval is deployed as follows: if for a given Outcome, the Time=  $t$ , then only events occurring in the time interval  $[t - \Delta T, t]$  are included
6. **Define statistics.** The numerical rule ('statistics') is given by a specific CP. Typical statistics may be chosen from the following list: Minimum, Maximum, Mean (Arithmetic Mean), Median, Mode, Range, Variance, Standard Deviation, Skewness, Kurtosis, Quartiles (First Quartile, Second Quartile, Third Quartile), Interquartile Range (IQR), Outliers, Percentiles, etc.

As a result, the final table for modelling has the structure as in Table 4: customer segments are simply copied (by a 'left join') procedure from the customer table, while the features are calculated for each given set of CPs as described above.

One can also introduce a naming convention for features, which merely lists the chosen values of CPs.

**Example 1.** Consider a simple example:

- A single Event table of historical emails, with a single Trigger\_segment variable with the type of the email campaign (with e.g. 4-5 possible values), and a single Boolean Response (TRUE if click),



**Table 4** The calculated features for modeling.

| Name               | Key   | Data type | Description                                               |
|--------------------|-------|-----------|-----------------------------------------------------------|
| Outcome.id         | TRUE  | Character | Unique id of the event                                    |
| Customer_segment.1 | FALSE | Character | Customer segmentation 1 (1 <sup>st</sup> segmententation) |
| ....               |       |           |                                                           |
| Customer_segment.l | FALSE | Character | Customer segmentation (l <sup>st</sup> segmententation)   |
| Feature.1          | FALSE | Numeric   | A feature                                                 |
| ....               |       |           |                                                           |
| Feature.n          | FALSE | Numeric   | A feature                                                 |
| Label              | FALSE | Boolean   | The outcome to be predicted and optimised                 |

- A single Customer\_segment (e.g. Gender),
- the Outcome table being identical to the Event table.

The feature denoted as

$$Gender\_Month\_MaxMean$$

takes into account all the emails sent to the customers of the same Gender, in a month prior to the time of the considered email, and has value 1 if a single email was clicked for each individual customer (otherwise 0), and finally all these values are averaged.

Importantly, the number of all possible features for all possible combinations of values of CPs is typically astronomical. The expert judgement and actual use case knowledge are required to reduce the number of constructed features to a typically still very large set commensurate to the size of the data set (typically in thousands or tens of thousands), but still much smaller than the number of all possible combinations.

We also note that further more complex features capturing synergies between different event tables (i.e. different channels), or different responses within the same table, could be construed by following the same principles.

## 5 Modeling and feature selection approach

In each use case, we start with typically more than 20,000 expert features that can be used for modeling and prediction of the binary classification models against the target (label). To choose optimal features, Feature validation and Feature selection steps are performed, as stated in [20] "...there is no group of filter methods that always outperforms all other methods".

The process of finding the best model [21] contains of four stages:

**Feature validation.** We run Monte Carlo simulation to benchmark all possible features against the target. Specifically, we run multiple iterations of a simple random forest classification against our target with small batch features (each iteration with 10-30 features selected randomly). This stage will give us an approximate overview of the quality of our features against the target.

**Feature selection.** This is the second step, where we choose the best fitted features, ranked by the score defined as

$$\text{score} = \text{importance} \cdot \sqrt{\text{auc}} \cdot \text{aucpr},$$

Where *importance* is the feature/variable importance (a higher score means that the specific feature will have a larger effect on the model that is being used to predict a certain variable), *auc* is the area under Receiver Operating Characteristic curve and *aucpr* is the area under Precision-Recall curve.

**Model training.** We now train different predictive models on a family of predictive model techniques, including but not limited to Logistics regression, Generalised additive models, Decision trees and random forests, Gradient boosting class of models, as well as various neural network architectures suitable to the data set size and structure.

**Model selection.** As the last step, each family of models is cross-validated by multiple train-test splits. The models are evaluated by comparing median of the AUC (or equivalently, Gini) metric on the test set, and the best model is chosen for deployment.

## 6 Examples

In this section we return to our examples from Section 3. In each case, we summarize the results of modeling by deploying the procedure described in this paper, on actual production data for some of the large corporations in Europe, United States and Africa, in life sciences and retail banking.

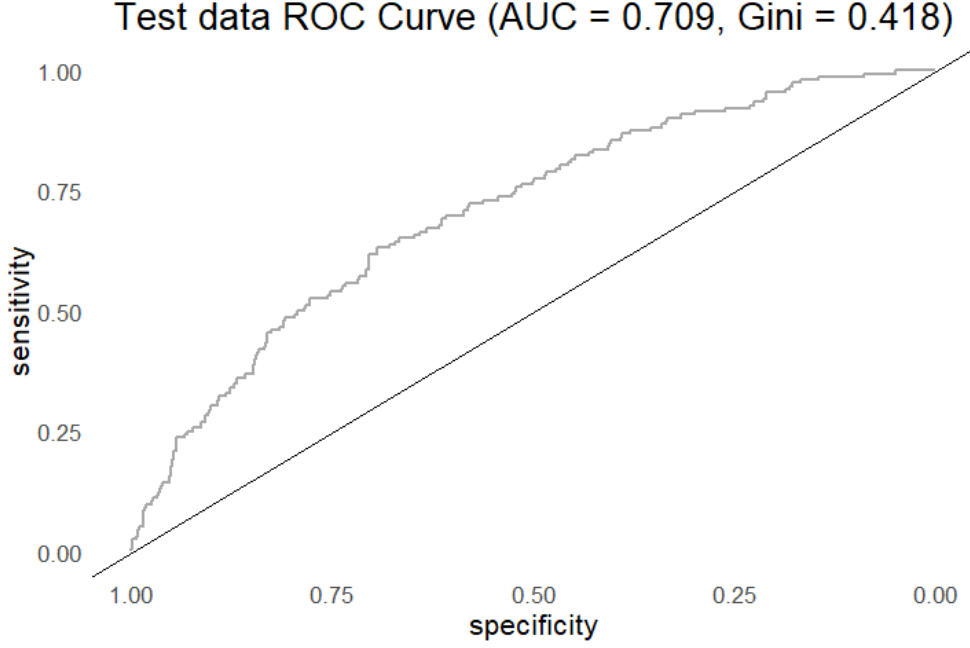
### Predicting engagement revisited

The predicting engagement model was developed on the set of data of actions of a Sales force in a B2B segment, targeting a rather small set of customers through multiple (physical, direct and other) channels.

We give here an example of a predictive model predicting likelihood of a click on a personalized email send by a Sales rep. The achieved AUC is on average 0.709 (corresponding to Gini 41.8 %). Considering that in this case the data set was relatively small (only 7,361 historical emails, with the 70 % / 30 % train/test split), the actual model is rather well performing. The key features of the data set are summarized in Table 5, the top 5 features by performance are listed in Table 6, and a sample of a typical ROC curve for this model is in Figure 2.

**Table 5** Example from practice: Predicting engagement, Summary

|                           |                              |
|---------------------------|------------------------------|
| No. of event tables       | 7 (different sales channels) |
| No. of outcomes           | 7,361                        |
| No. of generated features | 1,445                        |
| No. of used features      | 55                           |
| AUC range                 | [0.667, 0.751]               |

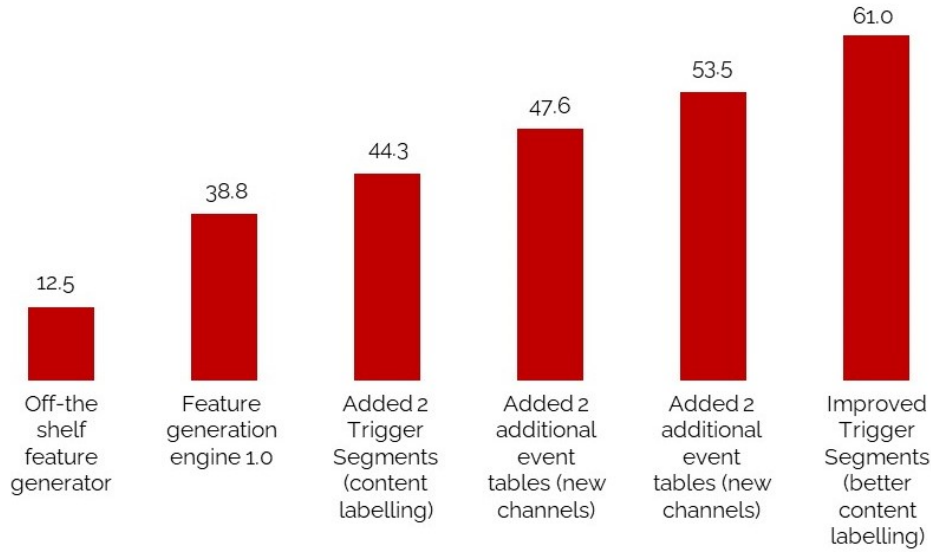


**Fig. 2** Example from practice: Predicting engagement, ROC curve

**Table 6** Top features with Importance: Predicting engagement

| Feature Name                          | Importance | Description                                                                             |
|---------------------------------------|------------|-----------------------------------------------------------------------------------------|
| email.click.all_template.all_mean     | 0.089      | Historical click rate on given email template                                           |
| email.click.segment_template.all_mean | 0.066      | Historical click rate on given template within the same customer segment                |
| email.open.customer_01y_dayspriorq25  | 0.056      | Number of days since the last opened email, the same customer, 25th quantile statistics |
| call.customer_01y_dayspriorq25        | 0.055      | Number of days since the last F2F visit, 25th quantile statistics, all within a year    |
| email.customer_01y_daysprior          | 0.054      | Number of days since the last email sent to the customer                                |

In addition, we give an example of the performance progression of the same use case, but on a larger data set (roughly 9 event tables, over 80,000 outcomes), in Figure 3. We note the three-fold improvement in performance of the model in the first step, by comparing the 'off-the-shelf' feature generation and by deploying the feature generation engine as described in this paper. (Further improvement is due to adding additional data sources and improved data engineering, e.g. content labelling; but was enabled by and driven by our feature generation approach).



**Fig. 3** Progression of accuracy performance of the predicting engagement model

### Propensity to buy revisited

We now revisit the example of the Propensity to buy model for a specific product category, on the data source of a large international bank with a digital platform, with the data source summarized in Table 7. The model performance with Gini on average 82% was achieved by deploying our feature generation methodology on 3 sources of transactional data (2 from the bank itself, and one from the digital platform), with most valuable information coming from the current account source, as shown in Table 8.

**Table 7** Example from practice: Propensity to buy

|                           |                                        |
|---------------------------|----------------------------------------|
| No. of event tables       | 3 (transactional data from 3 channels) |
| No. of outcomes           | 3,064,322                              |
| No. of generated features | 1,800                                  |
| No. of used features      | 30                                     |
| AUC range                 | [0.86, 0.91]                           |

### Credit scoring revisited

We now revisit the example of the behavioral credit scoring developed on roughly 22,000 historical loan applications, with 3 transactional data sources (Table 9) and one data source with demographic data. While the demographic data contribute to 2

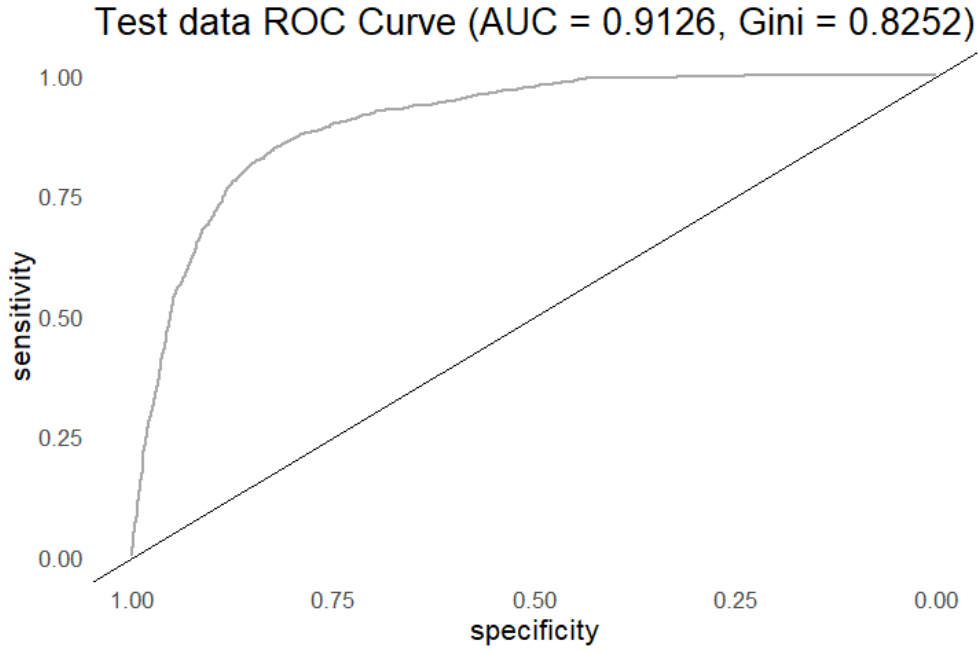


Fig. 4 Example from practice: Propensity to buy, ROC curve

Table 8 Top features with importance: Propensity to buy

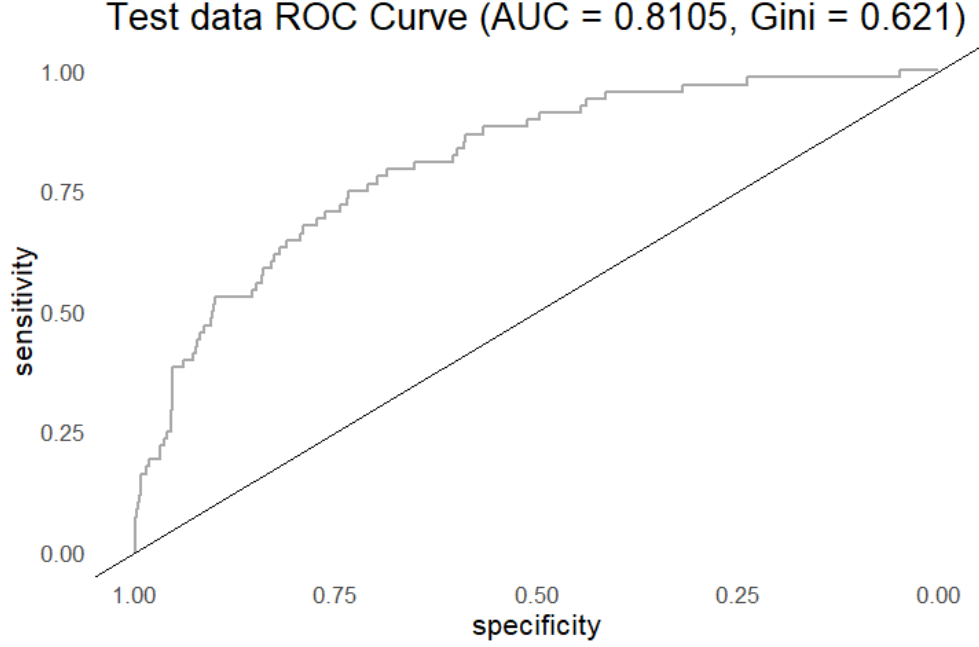
| Feature Name                      | Importance | Description                                                                            |
|-----------------------------------|------------|----------------------------------------------------------------------------------------|
| ca_amt_cst_out_03wk_sum           | 0.279      | Total amount of outgoing CA transactions in the last three weeks.                      |
| caMonthly_sumAmt_cst_out_06mn.q75 | 0.167      | 75th percentile of consecutive sums for the last 6 months of outgoing CA transactions. |
| caMonthly_sumAmt_cst_out_06mn.sum | 0.148      | Maximum of consecutive sums for the last 6 months of incoming CA transactions.         |
| ca_amt_cst_inc_03mn.mean          | 0.035      | Mean of incoming CA transactions in the last three months.                             |
| ca_amt_cst_inc_03mn.mean          | 0.035      | Mean of incoming CA transactions in the last three months.                             |

most important features (as seen in Table 10), most of other features (contributing cummulatively to over 80% of model performance) are of transactional nature, and are constructed by the feature generation procedure described in this paper.

The average accuracy of the model Gini of 62 % (see Figure 5 for a typical ROC curve) was in this case significantly better than the legacy credit scoring models (logistical regression on demographic data only).

**Table 9** Example from practice: Credit scoring

|                           |                                        |
|---------------------------|----------------------------------------|
| No. of event tables       | 3 (transactional data from 3 channels) |
| No. of outcomes           | 22,004                                 |
| No. of generated features | 26,637                                 |
| No. of used features      | 284                                    |
| AUC range                 | [0.7565, 0.8584]                       |

**Fig. 5** Example from practice: Credit scoring, ROC curve

## 7 Technical Approach

In this research, we combined R's [22] statistical tools with Apache Spark [23] through the `sparklyr` package [24] to create advanced machine learning models for binary classification. Our primary objective was to develop advanced machine learning models specifically tailored for binary classification tasks [25]. We leveraged the robust and scalable infrastructure provided by Amazon Web Services (AWS), which was used for managing the complexities and demands associated with large-scale binary classification challenges. More on AWS can be found [26]. The first technical step in the research was data preparation and exploration, utilizing the rich suite of tidyverse packages in R (see Appendix A for more information). The `dplyr` and `tidyr` packages were used efficiently for manipulating and structuring our data, ensuring it was optimally formatted for binary classification analysis. For visualisation, exploring the

**Table 10** Top features with importance: Credit scoring

| Feature Name                        | Importance | Description                                                                                          |
|-------------------------------------|------------|------------------------------------------------------------------------------------------------------|
| demo_overdraft_cst_account_all_iden | 0.043      | Account overdraft for that customer.                                                                 |
| demo_age_cst_all_all_iden           | 0.024      | Customer’s age.                                                                                      |
| ca95thPerc_amt_cst_all_all_lfq      | 0.023      | Last to first quarter ratio of 95th percentile for transaction amount relative to statement balance. |
| ca_amt_cst_inc_03mn_sum             | 0.022      | Sum of incoming CA transactions in the last three months.                                            |
| caMonthly_sdAmount_cst_all_all_sd   | 0.021      | Standard deviation of standard deviations of monthly transaction count.                              |

data and final output creation we used `ggplot2`, allowing us to capture complex patterns and relationships in binary classification. To handle big data challenges, we used `sparklyr` library for its seamless integration with Apache Spark, thereby enhancing our data processing capabilities.

The main part of our research was a complex process of developing and evaluating machine learning models for binary classification. Utilizing the `mlr3` [27] and `tidymodels` [28] frameworks, we were able to explore a diverse array of algorithms and techniques, fine-tuning our models to achieve optimal performance. The integration of additional R packages such as `caret` for model assessment, `data.table` for high-performance data manipulation, and `tibble` for improved data frame handling, significantly bolstered our capacity to develop highly accurate and efficient binary classification models.

The deployment phase of our binary classification models was characterized by a strategic utilization of various AWS services [26]. We used ECR for managing our Docker containers, providing a secure and efficient environment for our models. EC2 offered the necessary computational horsepower, essential for the rigorous demands of binary classification tasks. S3’s role in providing scalable and reliable data storage was indispensable, especially considering the voluminous nature of our datasets. Lastly, AWS Batch played a critical role in streamlining our batch processing workflows, ensuring that our binary classification models were not only scalable but also maintained high-performance standards in a production setting.

## 8 Deployment of models in production

We here summarise the key principles how the classification machine learning models developed in this section are deployed in actual production, on the Predicting engagement (production details for other model classes will be reported elsewhere).

In production, the trained models are typically deployed as follows:

*Step 1* An automated routine generates all possible, ‘allowed’ marketing messages to be sent to each customer, in the format of the Table 3, with unknown *Label*.

*Step 2* For each customer, ‘allowed’ marketing messages are filtered by deploying a certain family of business rules (e.g. do not send the same email twice to the same customer).

*Step 3* A trained classification model is deployed to all 'allowed' combinations of customer and marketing message, to calculate the probability of engagement (i.e. a number  $p$  in the interval  $[0, 1]$ ).

*Step 4* For each customer, only one message with the highest  $p$  is selected.

*Step 5* All messages with  $p > p_{\min}$  are selected for deployment, where the threshold  $p_{\min}$  is calculated by a separate cost-benefit optimisation procedure.

## 9 Conclusion

In this paper, we outlined a generic procedure for feature generation for a certain family of data sets occurring typically in the business practice. These data sets may summarise historical sales actions, behavioral data captured by a digital platform, or transactional data such as current account or credit card history. We have shown that the same, universal feature generation procedure successfully generates features for all these families of use cases, and can improve model accuracy 3-fold or more, as compared to when deploying standard, automated feature generation procedures available in R, Python or on platforms such as DataRobot.

Such improvement of model performance is not only theoretical. We have seen in our practice that this is critical for the predictive model to be 'good enough' for deployment and ultimate production.

Nevertheless, there are multiple further avenues for research. We list only a few here:

- We have described here only the key 6 steps in feature generation. As noted at the end of the section 4, one can generalize the specified procedure to capture synergies between different channels, or responses within the same channel, resulting with a more general algorithm, yet to be described in detail.
- We have noted that the expert knowledge is important, and is incorporated in our feature generation procedure in several steps, as described above. We intend to further investigate and report elsewhere the key decisions and principles, specific for each of listed use cases.
- Each of the three families of use cases described in this paper merits further in-depth research, and reporting on likely and achievable accuracy in different settings, depending on different parameters, such as the data set size, number of event tables, number of customers, the volume of demographic data, and other. We intend to report our further research on each use case separately in the future.
- One of the key data engineering steps for the Predicting engagement family of models is 'content labelling', i.e. categorising content so that it can be used as one or more 'Trigger Segments' in the feature generation algorithm described here. Again, we intend to report on our research on labelling of marketing content, in the context of overall content labelling and more generally Large Language Models research (for the Predicting engagement use case) separately.



## Acknowledgements

The research reported in this paper was supported entirely by Cantab Predictive Intelligence Ltd., Cambridge, UK.

## References

- [1] Shi, H., Li, H., Zhang, D., Cheng, C., Cao, X.: An efficient feature generation approach based on deep learning and feature selection techniques for traffic classification. *Computer Networks* **132**, 81–98 (2018) <https://doi.org/10.1016/j.comnet.2018.01.007>
- [2] Li, X., Zhang, X., Zhu, J., Mao, W., Sun, S., Wang, Z., Xia, C., Hu, B.: Depression recognition using machine learning methods with different feature generation strategies. *Artificial Intelligence in Medicine* **99**, 101696 (2019) <https://doi.org/10.1016/j.artmed.2019.07.004>
- [3] Lakshmanaprabu, S.K., Shankar, K., Ilayaraja, M., *et al.*: Random forest for big data classification in the internet of things using optimal features. *Int. J. Mach. Learn. Cyber* **10**, 2609–2618 (2019) <https://doi.org/10.1007/s13042-018-00916-z>
- [4] Yu, Z.-B., Zhang, M.-L.: Multi-label classification with label-specific feature generation: A wrapped approach. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **44**, 5199–5210 (2022) <https://doi.org/10.1109/TPAMI.2021.3070215>
- [5] Mumuni, A., Mumuni, F.: Automated data processing and feature engineering for deep learning and big data applications: a survey. *Journal of Information and Intelligence* (2024) <https://doi.org/10.1016/j.jiixd.2024.01.002>
- [6] Lou, R., Lalevic, D., Chambers, C., Zafar, H.M., Cook, T.S.: Automated detection of radiology reports that require follow-up imaging using natural language processing feature engineering and machine learning classification. *J Digit Imaging* **33**, 131–136 (2020) <https://doi.org/10.1007/s10278-019-00271-7>
- [7] Farahabadi, F.B., Vajargah, K.F., Farnoosh, R.: Dimension reduction big data using recognition of data features based on copula function and principal component analysis. *Advances in Mathematical Physics* **2021**, 8 (2021) <https://doi.org/10.1155/2021/9967368>
- [8] Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., Žídek, A., Potapenko, A., Bridgland, A., Meyer, C., Kohl, S.A.A., Ballard, A.J., Cowie, A., Romera-Paredes, B., Nikolov, S., Jain, R., Adler, J., Back, T., Petersen, S., Reiman, D., Clancy, E., Zielinski, M., Steinegger, M., Pacholska, M., Berghammer, T., Bodenstein, S., Silver, D., Vinyals, O., Senior, A.W., Kavukcuoglu, K., Kohli, P., Hassabis, D.: Highly accurate protein structure prediction with alphafold. *Nature* **596**(7873), 583–589 (2021)

<https://doi.org/10.1038/s41586-021-03819-2>

- [9] Evans, R., O'Neill, M., Pritzel, A., Antropova, N., Senior, A., Green, T., Žídek, A., Bates, R., Blackwell, S., Yim, J., Ronneberger, O., Bodenstein, S., Zielinski, M., Bridgland, A., Potapenko, A., Cowie, A., Tunyasuvunakool, K., Jain, R., Clancy, E., Kohli, P., Jumper, J., Hassabis, D.: Protein complex prediction with alphafold-multimer. *bioRxiv* (2022) <https://doi.org/10.1101/2021.10.04.463034>
- [10] Varadi, M., Anyango, S., Deshpande, M., Nair, S., Natassia, C., Yordanova, G., Yuan, D., Stroe, O., Wood, G., Laydon, A., Žídek, A., Green, T., Tunyasuvunakool, K., Petersen, S., Jumper, J., Clancy, E., Green, R., Vora, A., Lutfi, M., Figurnov, M., Cowie, A., Hobbs, N., Kohli, P., Kleywegt, G., Birney, E., Hassabis, D., Velankar, S.: AlphaFold Protein Structure Database: massively expanding the structural coverage of protein-sequence space with high-accuracy models. *Nucleic Acids Research* **50**(D1), 439–444 (2021) <https://doi.org/10.1093/nar/gkab1061>
- [11] Rawat, T., Dr. Khemchandani, V.: Feature engineering (fe) tools and techniques for better classification performance. *International Journal of Innovations in Engineering and Technology (IJJET)* **8**, 169–179 (2019) <https://doi.org/10.21172/ijiet.82.024>
- [12] Óskarsdóttir, M., Bravo, C., Sarraute, C., Vanthienen, J., Baesens, B.: The value of big data for credit scoring: Enhancing financial inclusion using mobile phone data and social network analytics. *Applied Soft Computing* **74**, 26–39 (2019) <https://doi.org/10.1016/j.asoc.2018.10.004>
- [13] Ma, L., Sun, B.: Machine learning and ai in marketing – connecting computing power to human insights. *International Journal of Research in Marketing* **37**(3), 481–504 (2020) <https://doi.org/10.1016/j.ijresmar.2020.04.005>
- [14] Islam, T., Meade, N., Carson, R.T., Louviere, J.J., Wang, J.: The usefulness of socio-demographic variables in predicting purchase decisions: Evidence from machine learning procedures. *Journal of Business Research* **151**, 324–338 (2022) <https://doi.org/10.1016/j.jbusres.2022.07.004>
- [15] Kamal, M., Aziz Bablu, T.: Machine Learning Models for Predicting Click-through Rates on social media: Factors and Performance Analysis. *International Journal of Applied Machine Learning and Computational Intelligence* **12**, 1–14 (2022)
- [16] Zhang, Y.: Prediction of Customer Propensity Based on Machine Learning. 2021 Asia-Pacific Conference on Communications Technology and Computer Science (ACCTCS), Shenyang, China, 2021, pp. 5-9, doi: 10.1109/ACCTCS52002.2021.00009 (2021)
- [17] Misiorek, P., Warmuz, J., Kaczmarek, D., Ciesielczyk, M.: Modeling user engagement profiles for detection of digital subscription propensity. In: Themistocleous,

- M., Papadaki, M. (eds.) Information Systems, pp. 55–68. Springer, Cham (2022)
- [18] Thomas, L., Crook, J., Edelman, D.: Credit Scoring and Its Applications. SIAM-Society for Industrial and Applied Mathematics, Philadelphia, PA, USA (2017)
  - [19] Gaudart, M., Slijepčević, S.: Open Banking current account information has the power to transform credit scoring and lending decisions. Credit Scoring and Credit Control Conference XVIII, Edinburgh, 2019., <https://www.crc.business-school.ed.ac.uk/sites/crc/files/2020-10/V81-Open-Banking-Current-Account-Information-Gaudart2-1.pdf> (2019)
  - [20] Bommert, A., Sun, X., Bischl, B., Rahnenführer, J., Lang, M.: Benchmark for filter methods for feature selection in high-dimensional classification data. Computational Statistics Data Analysis **143**, 106–839 (2020) <https://doi.org/10.1016/j.csda.2019.106839>
  - [21] Hastie, T., Tibshirani, R., Friedman, J.: Model assessment and selection. In: The Elements of Statistical Learning: Data Mining, Inference, and Prediction, pp. 219–259. Springer, New York (2009)
  - [22] R Core Team: R: A language and environment for statistical computing. Vienna, Austria: R Foundation for Statistical Computing. Retrieved from <https://www.R-project.org/> (2019)
  - [23] Salloum, S., Dautov, R., Chen, X., Peng, P.X., Huang, J.Z.: Big data analytics on apache spark. International Journal of Data Science and Analytics **1**, 145–164 (2016) <https://doi.org/10.1007/s41060-016-0027-9>
  - [24] Luraschi, J., Kuo, K., Ushey, K., Allaire, J., Falaki, H., Wang, L., Zhang, A., Li, Y.: sparklyr: R interface to apache spark. R package version 1(0) (2020)
  - [25] Chapman, C., Feit, E.M.: R For Marketing Research and Analytics. Springer, Springer Nature Switzerland AG 2019 (2019)
  - [26] Wittig, A., Wittig, M.: Amazon Web Services in Action: An In-depth Guide to AWS. Manning, New York (2023)
  - [27] Lang, M., *et al.*: mlr3: A modern object-oriented machine learning framework in r. Journal of Open Source Software **44**, 4 (2019) <https://doi.org/10.21105/joss.01903>
  - [28] Kuhn, M., Wickham, H.: Tidymodels: Easily install and load the 'tidymodels' packages. Retrieved from <https://CRAN.R-project.org/package=tidymodels> (2019)

## Appendix A Technical approach

### Data Preparation, Exploration, and Processing

**Tidyverse Packages:** The Tidyverse is a collection of R packages designed for data science. Each package is tailored to specific aspects of data manipulation, analysis, and visualization. In our study, we utilized a range of these packages to streamline our data workflows.

*dplyr*: For data manipulation, including filtering, sorting, and summarizing.

*tidyr*: Assists in transforming data into a tidy format.

*ggplot2*: Advanced data visualization capabilities.

*readr*: Fast and friendly data import from files.

*purrr*: Tools for working with functions and vectors.

*tibble*: Modernized data frames with enhanced features.

*stringr*: Simplifies string manipulation and processing.

*forcats*: For categorical variable handling and manipulation.

*lubridate*: Eases the work with dates and times.

*haven*: Simplifies the import of SAS, SPSS, and Stata file formats.

*readxl*: Provides capabilities to read Excel files.

*modelr*: Simplifies model fitting as part of a pipeline.

*broom*: Converts statistical analysis objects into tidy tibbles.

*dbplyr*: A dplyr back-end for databases for seamless data manipulation.

*rvest*: For web scraping and HTML data extraction.

*httr*: Provides tools for working with URLs and HTTP.

*jsonlite*: A robust and quick JSON parser and generator.

*xml2*: Tools for parsing and generating XML.

*reprex*: Helps create reproducible examples for troubleshooting.

*rlang*: Provides tools for programming with tidyverse packages.

### Data Processing with Sparklyr:

*sparklyr* : offers a dplyr-compatible interface to Apache Spark, enabling scalable data processing and handling large-scale datasets effectively.

### Machine Learning Model Development and Evaluation

#### mlr3 and Tidymodels:

*mlr3*: Provides a cohesive and extensible interface for various machine learning tasks, including model training, evaluation, and tuning.

*tidymodels*: A collection of packages for modeling and machine learning that share underlying design philosophies, grammar, and data structures.

#### Additional R Packages:

*caret*: Offers tools for model comparison and visualization, important for assessing model performance.

*data.table*: Enhances data manipulation and aggregation performance, particularly for large datasets.

*tibble*: Modernizes data frames for improved usability and consistency, useful for handling complex data outputs.

## Deployment and Scalability Using AWS

### AWS Services for Robust Deployment:

*ECR (Elastic Container Registry)*: Manages Docker container images, essential for reproducible and scalable model deployment.

*EC2 (Elastic Compute Cloud)*: Provides scalable computing capacity, hosting applications and models with flexible computational power.

*S3 (Simple Storage Service)*: Used for storing and retrieving large datasets, model artifacts, and other files, key for handling vast amounts of data.

*AWS Batch*: Streamlines batch computing, automating the deployment and management of batch processing jobs for optimal computational efficiency.

### Integration and Scalability:

The integration of R and Spark applications with AWS services ensures a seamless and scalable production environment, capable of handling large-scale data processing and complex computational tasks efficiently.

## Ensuring Reproducibility and Code Sharing

All analyses and model development processes are conducted using R scripts, shared alongside this paper. This practice ensures the integrity of our research and promotes further exploration and collaboration within the scientific community.